



1. Problem Title & Link

Title: LeetCode 1760 : Minimum Limit of Balls in a Bag

Link: [LeetCode 1760 - Minimum Limit of Balls in a Bag](#)

2. Problem Statement (Short Summary)

You are given:

nums[]

maxOperations

nums[i] represents the number of balls in a bag.

Operation:

- Choose one bag
- Split it into two bags

Goal:

Minimize the maximum number of balls in any bag after performing at most:

maxOperations

operations.

In simple terms:

Split bags intelligently so that the largest bag becomes as small as possible.

3. Examples (Input → Output)

Example 1

Input:

nums=[9]

maxOperations=2

Output:

3

Explanation:

Split:

9

→ 6 + 3

→ 3 + 3 + 3

Maximum balls:

3

**Example 2**

Input:

nums=[2,4,8,2]

maxOperations=4

Output:

2

Example 3

Input:

nums=[7,17]

maxOperations=2

Output:

7

4. Constraints

$1 \leq \text{nums.length} \leq 10^5$

$1 \leq \text{nums}[i] \leq 10^9$

$1 \leq \text{maxOperations} \leq 10^9$

Special restrictions:

- Need minimum possible maximum value
- Large constraints require optimization

5. Core Concept (Pattern / Topic)

Binary Search on Answer

6. Thought Process (Step-by-Step Explanation)**Brute Force**

Try every possible answer:

1,2,3,4,5...

For each value:

- Check if all bags can be converted

Time:

$O(\max(\text{nums}) \times n)$

Too slow.



Optimized Thinking

Observation:

Suppose:

$\text{maxBalls} = x$

We need to verify whether all bags can be converted so that no bag exceeds x .

For one bag:

$\text{balls} = 9$

$x = 3$

Required operations:

$(9-1)/3$

$=2$

Formula:

$\text{operations} +=$

$(\text{balls}-1)/x$

If total operations:

$\leq \text{maxOperations}$

Then x is valid.

Since validity changes monotonically:

Smaller $x \rightarrow$ harder

Larger $x \rightarrow$ easier

Binary search works.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

$\text{nums}=[9]$

$\text{maxOperations}=2$

Search:

1 ---5---9

$\text{mid}=5$

$\text{operations}=1$



Valid

Search left

1--2--4

mid=2

operations=4

Invalid

Search right

3

Answer=3

8. Pseudocode (Language Independent)

left=1

right=max(nums)

while left<right:

 mid=(left+right)/2

 operations=0

 for bag in nums:

 operations +=
 (bag-1)/mid

 if operations<=maxOperations:

 right=mid



else:

left=mid+1

return left

9. Code Implementation

Python

class Solution:

```
def minimumSize(  
    self,  
    nums,  
    maxOperations):
```

```
    left=1
```

```
    right=max(nums)
```

```
    while left<right:
```

```
        mid=(  
            left+right  
        )//2
```

```
        operations=0
```

```
        for bag in nums:
```

```
            operations += (  
                bag-1  
            )//mid
```

```
        if (  
            operations  
            <=maxOperations  
        ):
```



```
right=mid
```

```
else:
```

```
left=mid+1
```

```
return left
```

Java

```
class Solution {
```

```
    public int minimumSize(  
        int[] nums,  
        int maxOperations) {
```

```
        int left=1;
```

```
        int right=0;
```

```
        for(int num:nums)  
            right=Math.max(  
                right,  
                num  
            );
```

```
        while(left<right){
```

```
            int mid=  
                left+  
                (right-left)/2;
```

```
            long operations=0;
```

```
            for(int bag:nums){
```

```
                operations+=
```



```

        (bag-1)/mid;
    }

    if(
        operations
        <=maxOperations
    ){

        right=mid;
    }

    else{

        left=mid+1;
    }
}

return left;
}
}

```

10. Time & Space Complexity

Time Complexity:

$O(n \times \log(\max(\text{nums})))$

Reason:

- Binary search:

$\log(\max(\text{nums}))$

- Each iteration scans all bags

Space Complexity:

$O(1)$

11. Common Mistakes / Edge Cases

1. Wrong operation formula

Wrong:

bag/mid

Correct:



$(\text{bag}-1)/\text{mid}$

2. Integer overflow

Use:

long operations

in Java.

3. Incorrect binary search updates

4. Missing:

nums=[1]

case

12. Detailed Dry Run (Step-by-Step Table)

Input:

nums=[9]

maxOperations=2

left	right	mid	Operations	Action
1	9	5	1	Valid
1	5	3	2	Valid
1	3	2	4	Invalid

Final:

Answer=3

13. Common Use Cases (Real-Life / Interview)

Load balancing systems

Task splitting

Server request distribution

Resource allocation

14. Common Traps (Important!)

1. Using wrong split formula
2. Integer overflow
3. Brute forcing all values
4. Wrong binary search boundaries



15. Builds To (Related LeetCode Problems)

Progression:

- LC 1760 → Minimum Limit of Balls in a Bag
- LC 875 → Koko Eating Bananas
- LC 1011 → Capacity To Ship Packages Within D Days
- LC 1283 → Find Smallest Divisor Given Threshold
- LC 1482 → Minimum Days to Make m Bouquets

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(\max(\text{nums}) \times n)$	$O(1)$	Too slow
Binary Search on Answer	$O(n \log(\max(\text{nums})))$	$O(1)$	Optimal

17. Why This Solution Works (Short Intuition)

As the allowed maximum balls per bag increases, fewer operations are required. Since the validity changes monotonically, binary search efficiently finds the smallest possible valid value.

18. Variations / Follow-Up Questions

1. Return actual bag splits
2. Multiple split operations simultaneously
3. Minimize total operations instead
4. Variable split cost per bag
5. Minimize average balls per bag
- 6.